



3

Implementing a Backend Using Express, Mongoose ODM, and Jest

After learning the basics of Node.js and MongoDB, we will now put them into practice by building our first backend service using Express to provide a REST API, Mongoose **object data modeling (ODM)** to interface with MongoDB, and Jest to test our code. We will first learn how to structure a backend project using an architectural pattern. Then, we will create database schemas using Mongoose. Next, we will make service functions to interface with the database schemas and write tests for them using Jest. Then, we will learn what REST is and when it is useful. Finally, we provide a REST API and serve it using Express. At the end of this chapter, we will have a working backend service to be consumed by a front-end developed in the next chapter.

In this chapter, we are going to cover the following main topics:

- Designing a backend service
- Creating database schemas using Mongoose
- Developing and testing service functions
- Providing a REST API using Express

Technical requirements

Before we start, please install all requirements from [Chapter 1, Preparing for Full-stack Development](#), and [Chapter 2, Getting to Know Node.js and MongoDB](#).

The versions listed in those chapters are the ones used in the book. While installing a newer version should not be an issue, please note that certain steps might work differently on a newer version. If you are having an issue with the code and steps provided in this book, please try using the versions mentioned in *Chapters 1 and 2*.

You can find the code for this chapter on GitHub at

<https://github.com/PacktPublishing/Modern-Full-Stack-React-Projects/tree/main/ch3>.

If you cloned the full repository for the book, Husky may not find the `.git` directory when running `npm install`. In that case, just run `git init` in the root of the corresponding chapter folder.

The CiA video for the chapter can be found at:

<https://youtu.be/fHVVVn03rc>.

Designing a backend service

To design our backend service, we are going to use a variation of an existing architectural pattern called **model–view–controller (MVC)** pattern.

The MVC pattern consists of the following parts:

- **Model:** Handles data and basic data logic
- **Controller:** Controls how data is processed and displayed
- **View:** Displays the current state

In traditional full-stack applications, the backend would render and display the frontend completely, and an interaction would usually require a full-page refresh. The MVC architecture was designed mainly for such applications. However, in modern applications, the frontend is usually interactive and rendered in the backend only through server-side rendering. In modern applications, we thus often distinguish between the actual backend service(s) and the backend for frontend (which handles static site generation and server-side rendering):

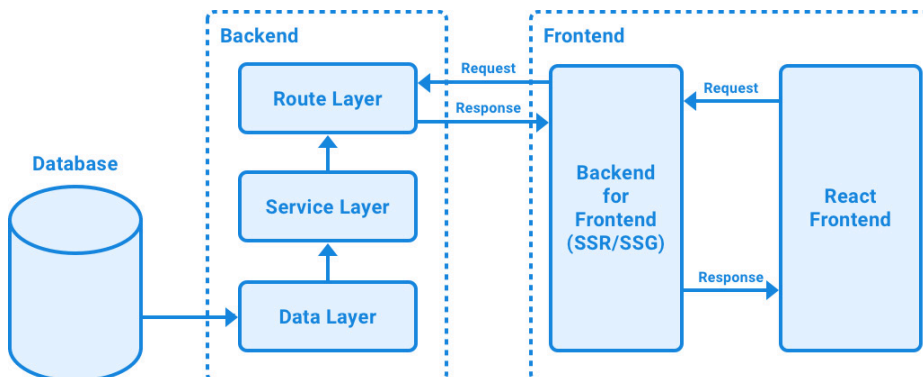


Figure 3.1 – A modern full-stack architecture, with a single backend service and a frontend with server-side rendering (SSR) and static-site generation (SSG)

For modern applications, the idea is that the backend service only deals with processing and serving requests and data and does not render the user interface anymore. Instead, we have a separate application that handles the frontend and server-side rendering of user interfaces specifically. To adapt to this change, we adjust the MVC architectural pattern to a data-service-route pattern for the backend service as follows:

- **Route layer:** Defines routes that consumers can access and handles user input by processing the request parameters and body and then calling service functions
- **Service layer:** Provides service functions, such as **create-read-update-delete (CRUD)** functions, which access the database through the data layer
- **Data layer:** Only deals with accessing the database and does basic validation to ensure that the database is consistent

This separation of concerns works best for services that only expose routes and do not deal with rendering user interfaces. Each layer in this pattern only deals with one step in processing the request.

After learning about the design of backend services, let's get started creating a folder structure reflecting what we have learned.

Creating the folder structure for our backend service

We are now going to create a folder structure for our backend service based on this pattern. Follow these steps:

1. First, copy the **ch2** folder to a new **ch3** folder to create a new folder for our backend service, as follows:

```
$ cp -R ch2 ch3
```

2. Open the new **ch3** folder in VS Code.
3. Edit the **.eslintrc.json** file and replace the **browser** env with the **node** and **es6** env, as follows:

```
"env": {  
  "node": true,  
  "es6": true  
},
```

4. Also, *remove* the **react** and **jsx-a11y** plugins from the **.eslintrc.json** file. We can also remove the React-related **settings** and **overrides** now by removing the highlighted lines:

```
"extends": [  
  "eslint:recommended",  
  "plugin:react/recommended",  
  "plugin:react/jsx-runtime",  
  "plugin:jsx-a11y/recommended",  
  "prettier"  
],  
"settings": {  
  "react": {  
    "version": "detect"  
  }  
},  
"overrides": [  
  {  
    "files": ["*.js", "*.jsx"]  
  }  
]  
]
```

5. *Delete* the **index.html** and **vite.config.js** files.
6. We can now also *remove* the **vite.config.js** file from the **.eslintignore** file:

```
dist/  
vite.config.js
```

7. *Delete* the **public**, **backend**, and **src** folders.
8. Open the **ch3** folder in VS Code, open a Terminal and run the following commands to remove **vite** and **react**:

```
$ npm uninstall --save react react-dom  
$ npm uninstall --save-dev vite @types/react \  
  @types/react-dom @vitejs/plugin-react \  
  eslint-plugin-jsx-a11y eslint-plugin-react
```

9. Edit the **package.json** file and *remove* the **dev**, **build**, and **preview** scripts from it:

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint src",  
  "preview": "vite preview",  
}
```

```
"prepare": "husky install"
},
```

10. Now, create a new **src/** folder, and within it, create **src/db/** (for the data layer), **src/services/** (for the services layer), and **src/routes/** (for the routes layer) folders.

Our first application is going to be a blog application. For such an application, we are going to need the API to be able to do the following:

- Get a list of posts
- Get a single post
- Create a new post
- Update an existing post
- Delete an existing post

To provide these functions, we first need to create a database schema to define what a blog post object should look like in our database. Then, we need service functions to handle CRUD functionality for blog posts. Finally, we are going to define our REST API to query, create, update, and delete blog posts.

Creating database schemas using Mongoose

Before we can get started defining the database schemas, we first need to set up Mongoose itself. Mongoose is a library that simplifies MongoDB object modeling by reducing the boilerplate code needed to interface with MongoDB. It also includes common business logic such as setting **createdAt** and **updatedAt** timestamps automatically and validation and type casting to keep the database state consistent.

Follow these steps to set up the **mongoose** library:

1. First, install the **mongoose** library:

```
$ npm install mongoose@8.0.2
```

2. Create a new **src/db/init.js** file and import **mongoose** there:

```
import mongoose from 'mongoose'
```

3. Define and export a function that will initialize the database connection:

```
export function initDatabase() {
```

4. First, we define **DATABASE_URL** to point to our local MongoDB instance running via Docker and specify **blog** as the database name:

```
const DATABASE_URL = 'mongodb://localhost:27017/blog'
```

The connection string is similar to what we used in the previous chapter when directly accessing the database via Node.js.

5. Then, add a listener to the **open** event on the Mongoose connection so that we can show a log message once we are connected to the database:

```
mongoose.connection.on('open', () => {  
  console.info('successfully connected to database:', DATABASE_URL)  
})
```

6. Now, use the **mongoose.connect()** function to connect to our MongoDB database and return the **connection** object:

```
const connection = mongoose.connect(DATABASE_URL)  
return connection  
}
```

7. Create a new **src/example.js** file and import and run the **initDatabase** function there:

```
import { initDatabase } from './db/init.js'  
initDatabase()
```

8. Run the **src/example.js** file using Node.js to see Mongoose successfully connecting to our database:

```
$ node src/example.js
```

As always, you can stop the server by pressing **Ctrl + C** in the Terminal.

We can see our log message being printed to the Terminal, so we know that Mongoose was able to successfully connect to our database! If there is an error, for example, because Docker (or the container) is not running, it will hang for a while and then throw an error about the connection be-

ing refused (**ECONNREFUSED**). In that case, make sure the Docker MongoDB container is running properly and can be connected to.

Defining a model for blog posts

After initializing the database, the first thing we should do is define the data structure for blog posts. Blog posts in our system should have a title, an author, contents, and some tags associated with the post. Follow these steps to define the data structure for blog posts:

1. Create a new **src/db/models/** folder.
2. Inside that folder, create a new **src/db/models/post.js** file, import the **mongoose** and the **Schema** classes:

```
import mongoose, { Schema } from 'mongoose'
```

3. Define a new schema for posts:

```
const postSchema = new Schema({
```

4. Now specify all properties of a blog post and the corresponding types. We have a required **title**, an **author**, and **contents**, which are all strings:

```
  title: { type: String, required: true },  
  author: String,  
  contents: String,
```

5. Lastly, we have **tags**, which are a string array:

```
  tags: [String],  
})
```

6. Now that we have defined the schema, we can create a Mongoose model from it by using the **mongoose.model()** function:

```
export const Post = mongoose.model('post', postSchema)
```

NOTE

*The first argument to **mongoose.model()** specifies the name of the collection. In our case, the collection will be called **posts** because we specified **post** as the name. In Mongoose models, we need to specify the name of the document in singular form.*

Now that we have defined the data structure and model for blog posts, we can start using it to create and query posts.

Using the blog post model

After creating our model, let's try using it! For now, we are simply going to access it in the **src/example.js** file because we have not defined any service functions or routes yet:

1. Import the **Post** model in the **src/example.js** file:

```
import { initDatabase } from './db/init.js'
import { Post } from './db/models/post.js'
```

2. The **initDatabase()** function we defined earlier is an **async** function, so we need to **await** it; otherwise, we would be attempting to access the database before we are connected to it:

```
await initDatabase()
```

3. Create a new blog post by calling **new Post()**, defining some example data:

```
const post = new Post({
  title: 'Hello Mongoose!',
  author: 'Daniel Bugl',
  contents: 'This post is stored in a MongoDB database using Mongoose.',
  tags: ['mongoose', 'mongodb'],
})
```

4. Call **.save()** on the blog post to save it to the database:

```
await post.save()
```

5. Now we can use the **.find()** function to list all posts, and log the result:

```
const posts = await Post.find()
console.log(posts)
```

6. Run the example script to see our post being inserted and listed:

```
$ node src/example.js
```

You will get the following result after running the preceding script:


```

○ → ~/D/F/ch3 ↗ main+ > node src/test.js
successfully connected to database: mongodb://localhost:27017/ch3
[
  {
    _id: new ObjectId("64281a742bd96957bf8bde51"),
    title: 'Hello Mongoose!',
    author: 'Daniel Bugl',
    contents: 'This post is stored in a MongoDB database using Mongoose.',
    tags: [ 'mongoose', 'mongodb' ],
    __v: 0
  }
]

```

Figure 3.2 – Our first document inserted via Mongoose!

As you can see, using Mongoose is very similar to using MongoDB directly. However, it offers us some wrappers around models for convenience, making it easier to deal with documents.

Defining creation and last update dates in the blog post

You may have noticed that we have not added any dates to our blog post. So, we do not know when a blog post is created or when it was last updated. Mongoose makes implementing such functionality simple, let's try it out now:

1. Edit the `src/db/models/post.js` file and add a second argument to the new `Schema()` constructor. The second argument specifies options for the schema. Here, we set the `timestamps: true` setting:

```

const postSchema = new Schema(
  {
    title: String,
    author: String,
    contents: String,
    tags: [String],
  },
  { timestamps: true },
)

```

2. Now all we need to do is create a new blog post by running the example script, and we will see that the last post inserted now has `createdAt` and `updatedAt` timestamps:

```
$ node src/example.js
```

3. To see if the `updatedAt` timestamp works, let's try updating the created blog post by using the `findByIdAndUpdate` method. Save the result of `await post.save()` in a `createdPost` constant, then add the following

code close to the end of the `src/example.js` file, before the `Post.find()` call:

```
const createdPost = await post.save()
await Post.findByIdAndUpdate(createdPost._id, {
  $set: { title: 'Hello again, Mongoose!' },
})
```

4. Run the server again to see the blog posts being updated:

```
$ node src/example.js
```

You will get three posts, and the last one of them now looks as follows:

```
{
  _id: new ObjectId("64281e304e7d65f85ce07e1f"),
  title: 'Hello again, Mongoose!',
  author: 'Daniel Bugl',
  contents: 'This post is stored in a MongoDB database using Mongoose.',
  tags: [ 'mongoose', 'mongodb' ],
  createdAt: 2023-04-01T12:06:08.303Z,
  updatedAt: 2023-04-01T12:06:08.312Z,
  __v: 0
}
```

Figure 3.3 – Our updated document with the automatically updated timestamps

As we can see, using Mongoose makes dealing with MongoDB documents much more convenient! Now that we have defined our database model, let's start developing (and writing tests for) service functions!

Developing and testing service functions

Up until now, we have always been testing code by putting it in the `src/example.js` file. Now, we are going to write some service functions and learn how to write actual tests for them by using Jest.

Setting up the test environment

First, we are going to set up our test environment by following these steps:

1. Install `jest` and `mongodb-memory-server` as dev dependencies:

```
$ npm install --save-dev jest@29.7.0 \
```

```
mongodb-memory-server@9.1.1
```

Jest is a test runner used to define and execute unit tests. The **mongodb-memory-server** library allows us to spin up a fresh instance of a MongoDB database, storing data only in memory, so that we can run our tests on a fresh database instance.

2. Create a **src/test/** folder to put the setup for our tests in.
3. In this folder, create a **src/test/globalSetup.js** file, where we will import **MongoMemoryServer** from the previously installed library:

```
import { MongoMemoryServer } from 'mongodb-memory-server'
```

4. Now define a **globalSetup** function, which creates a memory server for MongoDB:

```
export default async function globalSetup() {  
  const instance = await MongoMemoryServer.create({
```

5. When creating the **MongoMemoryServer**, set the binary version to **6.0.4**, which is the same version that we installed for our Docker container:

```
    binary: {  
      version: '6.0.4',  
    },  
  })
```

6. We will store the MongoDB instance as a global variable to be able to access it later in the **globalTeardown** function:

```
global.__MONGOINSTANCE = instance
```

7. We will also store the URL to connect to our test instance in the **DATABASE_URL** environment variable:

```
process.env.DATABASE_URL = instance.getUri()  
}
```

8. Edit **src/db/init.js** and adjust the **DATABASE_URL** to come from the environment variable so that our tests will be using the correct database:

```
export function initDatabase() {  
  const DATABASE_URL = process.env.DATABASE_URL
```

9. Additionally, create a **src/test/globalTeardown.js** file to stop the MongoDB instance when our tests are finished and add the following code inside it:

```
export default async function globalTeardown() {
  await global.__MONGOINSTANCE.stop()
}
```

10. Now, create a **src/test/setupFileAfterEnv.js** file. Here, we will define a **beforeAll** function to initialize our database connection in Mongoose before all tests run and an **afterAll** function to disconnect from the database after all tests finish running:

```
import mongoose from 'mongoose'
import { beforeAll, afterAll } from '@jest/globals'
import { initDatabase } from '../db/init.js'
beforeAll(async () => {
  await initDatabase()
})
afterAll(async () => {
  await mongoose.disconnect()
})
```

11. Then, create a new **jest.config.json** file in the root of our project where we will define the config for our tests. In the **jest.config.json** file, we first set the test environment to **node**:

```
{
  "testEnvironment": "node",
```

12. Next, tell Jest to use the **globalSetup**, **globalTeardown**, and **setupFileAfterEnv** files we created earlier:

```
"globalSetup": "<rootDir>/src/test/globalSetup.js",
"globalTeardown": "<rootDir>/src/test/globalTeardown.js",
"setupFilesAfterEnv": ["<rootDir>/src/test/setupFileAfterEnv.js"]
}
```

NOTE

*In this case, **<rootDir>** is a special string that automatically gets resolved to the root directory by Jest. You do not need to manually fill in a root directory here.*

13. Finally, edit the **package.json** file and add a **test** script, which will run Jest:

```
"scripts": {
  "test": "NODE_OPTIONS=--experimental-vm-modules jest",
  "lint": "eslint src",
  "prepare": "husky install"
},
```

NOTE

*At the time of writing, the JavaScript ecosystem is still in the process of moving to the **ECMAScript module (ESM)** standard. In this book, we already use this new standard. However, Jest does not support it yet by default, so we need to pass the **--experimental-vm-modules** option when running Jest.*

14. If we attempt running this script now, we will see that there are no tests found, but we can still see that Jest is set up and working properly:

```
$ npm test
```

```
❌ → ~/D/F/ch3 ✎ main± > npm run test

> ch3@0.0.0 test
> NODE_OPTIONS=--experimental-vm-modules jest

No tests found, exiting with code 1
Run with `--passWithNoTests` to exit with code 0
In /Users/dan/Development/Full-Stack-React-Projects/ch3
  14 files checked.
    testMatch: **/__tests__/**/*.[jt]s?(x), **/?(*.)+(spec|test).[jt]s?(x) - 0 matches
    testPathIgnorePatterns: /node_modules/ - 14 matches
    testRegex: - 0 matches
    Pattern: - 0 matches
```

Figure 3.4 – Jest is set up successfully, but we have not defined any tests yet

Now that our test environment is set up, we can start writing our service functions and unit tests. It is always a good idea to write unit tests right after writing service functions, as it means we will be able to debug them right away while still having their intended behavior fresh in our minds.

Writing our first service function: createPost

For our first service function, we are going to make a function to create a new post. We can then write tests for it by verifying that the create function creates a new post with the specified properties. Follow these steps:

1. Create a new **src/services/posts.js** file.
2. In the **src/services/posts.js** file, first import the **Post** model:

```
import { Post } from '../db/models/post.js'
```

3. Define a new **createPost** function, which takes an object with **title**, **author**, **contents**, and **tags** as arguments and creates and returns a new post:

```
export async function createPost({ title, author, contents, tags }) {  
  const post = new Post({ title, author, contents, tags })  
  return await post.save()  
}
```

We specifically listed all properties that we want the user to be able to provide here instead of simply passing the whole object to the **new Post()** constructor. While we need to type more code this way, it allows us to have control over which properties a user should be able to set. For example, if we later add permissions to the database models, we may be accidentally allowing users to set those permissions here, if we forget to exclude those properties. For those security reasons, it is always good practice to have a list of allowed properties instead of simply passing down the whole object.

After writing our first service function, let's continue by writing test cases for it.

Defining test cases for the createPost service function

To test if the **createPost** function works as expected, we are going to define unit test cases for it using Jest:

1. Create a new **src/__tests__** folder, which will contain all test definitions.

NOTE

*Alternatively, test files can also be co-located with the related files that they are testing. However, in this book, we use the **__tests__** directory to make*

it easier to distinguish tests from other files.

2. Create a new `src/__tests__/posts.test.js` file for our tests related to posts. In this file, start by importing `mongoose` and the `describe`, `expect`, and `test` functions from `@jest/globals`:

```
import mongoose from 'mongoose'
import { describe, expect, test } from '@jest/globals'
```

3. Also import the `createPost` function from our services and the `Post` model:

```
import { createPost } from '../services/posts.js'
import { Post } from '../db/models/post.js'
```

4. Then, use the `describe()` function to define a new test. This function describes a group of tests. We can call our group `creating posts`:

```
describe('creating posts', () => {
```

5. Inside the group, we will define a test by using the `test()` function. We can pass an `async` function here to be able to use `async/await` syntax. We call the first test `creating posts with all parameters should succeed`:

```
test('with all parameters should succeed', async () => {
```

6. Inside this test, we will use the `createPost` function to create a new post with some parameters:

```
const post = {
  title: 'Hello Mongoose!',
  author: 'Daniel Bugl',
  contents: 'This post is stored in a MongoDB database using Mongoose.',
  tags: ['mongoose', 'mongodb'],
}
const createdPost = await createPost(post)
```

7. Then, verify that it returns a post with an ID by using the `expect()` function from Jest and the `toBeInstanceOf` matcher to verify that it is an `ObjectId`:

```
expect(createdPost._id).toBeInstanceOf(mongoose.Types.ObjectId)
```

8. Now use Mongoose directly to find the post with the given ID:

```
const foundPost = await Post.findById(createdPost._id)
```

9. We **expect()** the **foundPost** to equal an object containing at least the properties of the original post object we defined. Additionally, we expect the created post to have **createdAt** and **updatedAt** timestamps:

```
expect(foundPost).toEqual(expect.objectContaining(post))
expect(foundPost.createdAt).toBeInstanceOf(Date)
expect(foundPost.updatedAt).toBeInstanceOf(Date)
})
```

10. Additionally, define a second test, called **creating posts without title should fail**. As we defined the **title** to be required, it should not be possible to create a post without one:

```
test('without title should fail', async () => {
  const post = {
    author: 'Daniel Bugl',
    contents: 'Post with no title',
    tags: ['empty'],
  }
})
```

11. Use a **try/catch** construct to catch the error and **expect()** the error to be a Mongoose **ValidationError**, which tells us that the title is required:

```
try {
  await createPost(post)
} catch (err) {
  expect(err).toBeInstanceOf(mongoose.Error.ValidationError)
  expect(err.message).toContain('`title` is required')
}
})
```

12. Finally, make a test called **creating posts with minimal parameters should succeed** and only enter the **title**:

```
test('with minimal parameters should succeed', async () => {
  const post = {
    title: 'Only a title',
  }
  const createdPost = await createPost(post)
  expect(createdPost._id).toBeInstanceOf(mongoose.Types.ObjectId)
})
})
```


13. Now that we have defined our tests, run the script we defined earlier:

```
$ npm test
```

As we can see, using unit tests we can do isolated tests on our service functions without having to define and manually access routes or write some manual test setups. These tests also have the added advantage that when we change code later, we can ensure that the previously defined behavior did not change by re-running the tests.

Defining a function to list posts

After defining a function to create posts, we are now going to define an internal **listPosts** function, which allows us to query posts and define a sort order. Then, we are going to use this function to define **listAllPosts**, **listPostsByAuthor**, and **listPostsByTag** functions:

1. Edit the **src/services/posts.js** file and define a new function at the end of the file.

The function accepts a **query** and an **options** argument (with **sortBy** and **sortOrder** properties). With **sortBy**, we can define which field we want to sort by, and the **sortOrder** argument allows us to specify whether posts should be sorted in ascending or descending order. By default, we list all posts (empty object as query) and show the newest posts first (sorted by **createdAt**, in **descending** order):

```
async function listPosts(  
  query = {},  
  { sortBy = 'createdAt', sortOrder = 'descending' } = {},  
) {
```

2. We can use the **.find()** method from our Mongoose model to list all posts, passing an argument to sort them:

```
  return await Post.find(query).sort({ [sortBy]: sortOrder })  
}
```

3. Now we can define a function to list all posts, which simply passes an empty object as query:

```
export async function listAllPosts(options) {  
  return await listPosts({}, options)  
}
```

4. Similarly, we can create a function to list all posts by a certain author by passing **author** to the query object:

```
export async function listPostsByAuthor(author, options) {  
  return await listPosts({ author }, options)  
}
```

5. Lastly, define a function to list posts by tag:

```
export async function listPostsByTag(tags, options) {  
  return await listPosts({ tags }, options)  
}
```

In MongoDB, we can simply match strings in an array by matching the string as if it was a single value, so all we need to do is add a query for **tags: 'nodejs'**. MongoDB will then return all documents that have a **'nodejs'** string in their **tags** array.

NOTE

The **{ [variable]: ... }** operator resolves the string stored in the **variable** to a key name for the created object. So, if our variable contains **'createdAt'**, the resulting object will be **{ createdAt: ... }**.

After defining the list post function, let's also write test cases for it.

Defining test cases for list posts

Defining test cases for list posts is similar to create posts. However, we now need to create an initial state where we already have some posts in the database to be able to test the list functions. We can do this by using the **beforeEach()** function, which executes some code before each test case is executed. We can use the **beforeEach()** function for a whole test file or only run it for each test inside a **describe()** group. In our case, we are going to define it for the whole file, as the sample posts will come in handy when we test the delete post function later:

1. Edit the **src/__tests__/posts.js** file, adjust the **import** statement to import the **beforeEach** function from **@jest/globals** and import the various functions to list posts from our services:

```
import { describe, expect, test, beforeEach } from '@jest/globals'  
import { createPost,  
  listAllPosts,
```

```

    listPostsByAuthor,
    listPostsByTag,
  } from '../services/posts.js'

```

2. At the end of the file, define an array of sample posts:

```

const samplePosts = [
  { title: 'Learning Redux', author: 'Daniel Bugl', tags: ['redux'] },
  { title: 'Learn React Hooks', author: 'Daniel Bugl', tags: ['react'] },
  {
    title: 'Full-Stack React Projects',
    author: 'Daniel Bugl',
    tags: ['react', 'nodejs'],
  },
  { title: 'Guide to TypeScript' },
]

```

3. Now, define an empty array, which will be populated with the created posts. Then, define a **beforeEach** function, which first clears all posts from the database and clears the array of created sample posts and then creates the sample posts in the database again for each of the posts defined in the array earlier. This ensures that we have a consistent state of the database before each test case runs and that we have an array to compare against when testing the list post functions:

```

let createdSamplePosts = []
beforeEach(async () => {
  await Post.deleteMany({})
  createdSamplePosts = []
  for (const post of samplePosts) {
    const createdPost = new Post(post)
    createdSamplePosts.push(await createdPost.save())
  }
})

```

To ensure that our unit tests are modular and independent from each other, we insert posts into the database directly by using Mongoose functions (instead of the **createPost** function).

4. Now that we have some sample posts ready, let's write our first test case, which should simply list all posts. We will define a new test group for **listing posts** and a test to verify that all sample posts are listed by the **listAllPosts()** function:

```

describe('listing posts', () => {
  test('should return all posts', async () => {
    const posts = await listAllPosts()

```

```
expect(posts.length).toEqual(createdSamplePosts.length)
})
```

5. Next, make a test that verifies that the default sort order shows newest posts first. We sort the **createdSamplePosts** array manually by **createdAt** (descending) and then compare the sorted dates to those returned from the **listAllPosts()** function:

```
test('should return posts sorted by creation date descending by default', async
const posts = await listAllPosts()
const sortedSamplePosts = createdSamplePosts.sort(
  (a, b) => b.createdAt - a.createdAt,
)
expect(posts.map((post) => post.createdAt)).toEqual(
  sortedSamplePosts.map((post) => post.createdAt),
)
})
```

NOTE

The **.map()** function applies a function to each element of an array and returns the result. In our case, we select the **createdAt** property from all elements of the array. We cannot directly compare the arrays with each other because Mongoose returns documents with a lot of additional information in hidden metadata, which Jest will attempt to compare.

6. Additionally, define a test case where the **sortBy** value is changed to **updatedAt**, and the **sortOrder** value is changed to **ascending** (showing oldest updated posts first):

```
test('should take into account provided sorting options', async () => {
  const posts = await listAllPosts({
    sortBy: 'updatedAt',
    sortOrder: 'ascending',
  })
  const sortedSamplePosts = createdSamplePosts.sort(
    (a, b) => a.updatedAt - b.updatedAt,
  )
  expect(posts.map((post) => post.updatedAt)).toEqual(
    sortedSamplePosts.map((post) => post.updatedAt),
  )
})
```

7. Then, add a test to ensure that listing posts by author works:

```
test('should be able to filter posts by author', async () => {
```

```
const posts = await listPostsByAuthor('Daniel Bugl')
expect(posts.length).toBe(3)
})
```

NOTE

We are controlling the test environment by creating a specific set of sample posts before each test case runs. We can make use of this controlled environment to simplify our tests. As we already know that there are only three posts with that author, we can simply check if the function returned exactly three posts. Doing so keeps our tests simple, and they are still safe because we control the environment completely.

8. Finally, add a test to verify that listing posts by tag works:

```
test('should be able to filter posts by tag', async () => {
  const posts = await listPostsByTag('nodejs')
  expect(posts.length).toBe(1)
})
})
```

9. Run the tests again and watch them all pass:

```
$ npm test
```

```
PASS src/__tests__/posts.test.js
  creating posts
    ✓ with all parameters should succeed (16 ms)
    ✓ without title should fail (5 ms)
    ✓ with minimal parameters should succeed (6 ms)
  listing posts
    ✓ should return all posts (4 ms)
    ✓ should return posts sorted by creation date descending by default (4 ms)
    ✓ should take into account provided sorting options (4 ms)
    ✓ should be able to filter posts by author (3 ms)
    ✓ should be able to filter posts by tag (4 ms)

Test Suites: 1 passed, 1 total
Tests:      8 passed, 8 total
Snapshots:  0 total
Time:       0.349 s, estimated 1 s
Ran all test suites.
```

Figure 3.5 – All our tests passing successfully!

As we can see, for some tests, we need to prepare an initial state. In our case, we only had to create some posts, but this initial state may become more sophisticated. For example, on a more advanced blogging platform, it may be necessary to create a user account first, then create a blog on the platform, and then create blog posts for that blog. In that case, we could create test utility functions, such as `createTestUser`, `createTest-`

Blog, **createTestPost** and import them in our tests. We can then use these functions in **beforeEach()** across multiple test files instead of manually doing it every single time. Depending on your application structure, different test utility functions may be needed, so feel free to define them as you see fit.

After defining the test cases for the list posts function, let's continue by defining the get single post, update post, and delete post functions.

Defining the get single post, update and delete post functions

The get single post, update and delete post functions can be defined very similarly to the list posts function. Let's do that quickly now:

1. Edit the **src/services/posts.js** file and define a **getPostById** function as follows:

```
export async function getPostById(postId) {  
  return await Post.findById(postId)  
}
```

It may seem a bit trivial to define a service function that just calls **Post.findById**, but it is good practice to define it anyway. Later, we may want to add some additional restrictions, such as access control. Having the service function allows us to change it only in one place and we do not have to worry about forgetting to add it somewhere. Another benefit is that if we, for example, want to change the database provider later, the developer only needs to worry about getting the service functions working again, and they can be verified with the test cases.

2. In the same file, define the **updatePost** function. It will take an ID of an existing post, and an object of parameters to be updated. We are going to use the **findOneAndUpdate** function from Mongoose, together with the **\$set** operator, to change the specified parameters. As a third argument, we provide an options object with **new: true** so that the function returns the modified object instead of the original:

```
export async function updatePost(postId, { title, author, contents, tags }) {  
  return await Post.findOneAndUpdate(  
    { _id: postId },  
    { $set: { title, author, contents, tags } },  
    { new: true },
```

```
)
}
```

3. In the same file, also define a **deletePost** function, which simply takes the ID of an existing post and deletes it from the database:

```
export async function deletePost(postId) {
  return await Post.deleteOne({ _id: postId })
}
```

TIP

*Depending on your application, you may want to set a **deletedOn** time-stamp instead of deleting it right away. Then, set up a function that gets all documents that have been deleted for more than 30 days and delete them. Of course, this means that we need to always filter out already deleted posts in the **listPosts** function and that we need to write test cases for this behavior!*

4. Edit the **src/__tests__/posts.js** file and import the **getPostById** function:

```
  getPostById,
} from '../services/posts.js'
```

5. Add tests for getting a post by ID and failing to get a post because the ID did not exist in the database:

```
describe('getting a post', () => {
  test('should return the full post', async () => {
    const post = await getPostById(createdSamplePosts[0]._id)
    expect(post.toObject()).toEqual(createdSamplePosts[0].toObject())
  })
  test('should fail if the id does not exist', async () => {
    const post = await getPostById('000000000000000000000000')
    expect(post).toEqual(null)
  })
})
```

In the first test, we use **.toObject()** to convert the Mongoose object with all its internal properties and metadata to a **plain old JavaScript object (POJO)** so that we can compare it to the sample post object by comparing all properties.

6. Next, import the **updatePost** function:

```
    updatePost,
  } from '../services/posts.js'
```

7. Then, add tests for updating a post successfully. We add one test to verify that the specified property was changed and another test to verify that it does not interfere with other properties:

```
describe('updating posts', () => {
  test('should update the specified property', async () => {
    await updatePost(createdSamplePosts[0]._id, {
      author: 'Test Author',
    })
    const updatedPost = await Post.findById(createdSamplePosts[0]._id)
    expect(updatedPost.author).toEqual('Test Author')
  })
  test('should not update other properties', async () => {
    await updatePost(createdSamplePosts[0]._id, {
      author: 'Test Author',
    })
    const updatedPost = await Post.findById(createdSamplePosts[0]._id)
    expect(updatedPost.title).toEqual('Learning Redux')
  })
})
```

8. Additionally, add a test to ensure the **updatedAt** timestamp was updated. To do so, first convert the **Date** objects to numbers by using **.getTime()**, and then we can compare them by using the **expect(...).toBeGreaterThan(...)** matcher:

```
test('should update the updatedAt timestamp', async () => {
  await updatePost(createdSamplePosts[0]._id, {
    author: 'Test Author',
  })
  const updatedPost = await Post.findById(createdSamplePosts[0]._id)
  expect(updatedPost.updatedAt.getTime()).toBeGreaterThan(
    createdSamplePosts[0].updatedAt.getTime(),
  )
})
```

9. Also add a failing test to see if the **updatePost** function returns **null** when no post with a matching ID was found:

```
test('should fail if the id does not exist', async () => {
  const post = await updatePost('000000000000000000000000', {
    author: 'Test Author',
  })
  expect(post).toEqual(null)
})
```



```
    })
  })
}
```

10. Lastly, import the **deletePost** function:

```
    deletePost,
  } from '../services/posts.js'
```

11. Then, add tests for successful and unsuccessful deletes by checking if the post was deleted and verifying the returned **deletedCount**:

```
describe('deleting posts', () => {
  test('should remove the post from the database', async () => {
    const result = await deletePost(createdSamplePosts[0]._id)
    expect(result.deletedCount).toEqual(1)
    const deletedPost = await Post.findById(createdSamplePosts[0]._id)
    expect(deletedPost).toEqual(null)
  })
  test('should fail if the id does not exist', async () => {
    const result = await deletePost('000000000000000000000000')
    expect(result.deletedCount).toEqual(0)
  })
})
```

12. Finally, run all tests again; they should all pass:

```
$ npm test
```

Writing tests for service functions may be tedious, but it will save us a lot of time in the long run. Adding additional functionality later, such as access control, may change the basic behavior of the service functions. By having the unit tests, we can ensure that we do not break existing behavior when adding new functionality.

Using the Jest VS Code extension

Up until now, we have run our tests by using Jest via the Terminal. There is also a Jest extension for VS Code, which we can use to make running tests more visual. The extension is especially helpful for larger projects where we have many tests in multiple files. Additionally, the extension can automatically watch and re-run tests if we change the definitions. We can install the extension as follows:

1. Go to the **Extensions** tab in the VS Code sidebar.
2. Enter **Orta.vscode-jest** in the search box to find the Jest extension.

3. Install the extension by pressing the **Install** button.
4. Now go to the newly added test icon on the sidebar (it should be a chemistry flask icon):

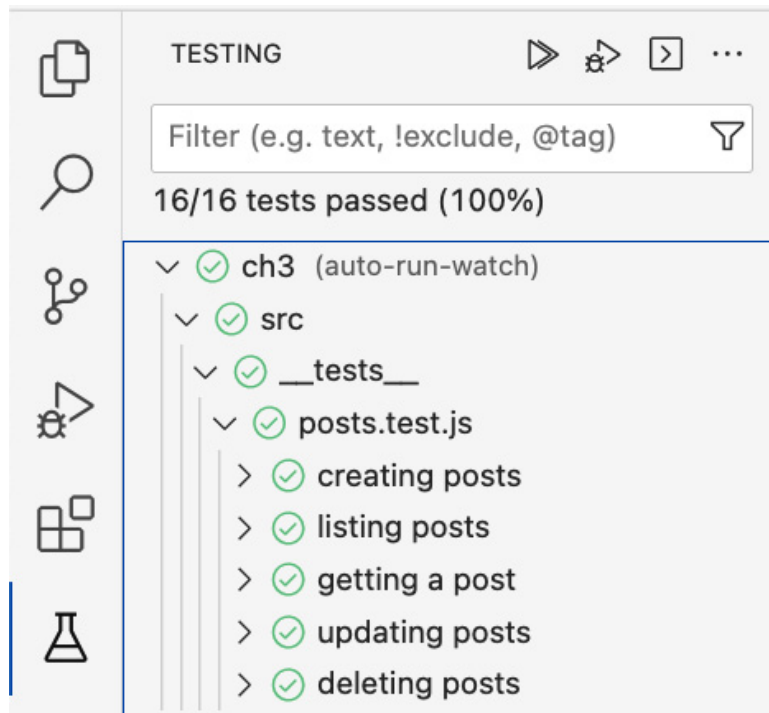


Figure 3.6 – The Testing tab in VS Code provided by the Jest extension

The Jest extension provides us an overview of all tests that we have defined. We can hover over them and press on the **Play** icon to re-run a specific test. By default, the Jest extension enables **auto-run-watch** (as can be seen in *Figure 3.6*). If **auto-run-watch** is enabled, the extension will re-run tests automatically when test definition files are saved. That's pretty handy!

Now that we have defined and tested our service functions, we can start using them when defining routes, which we are going to do next!

Providing a REST API using Express

Having our data and service layers set up, we have a good framework for being able to write our backend. However, we still need an interface that lets users access our backend. This interface will be a **representational state transfer (REST)** API. A REST API provides a way to access our server via HTTP requests, which we can make use of when we develop our frontend.

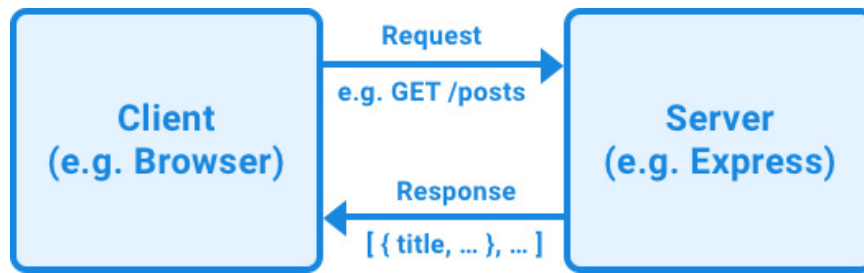


Figure 3.7 – The interaction between client and server using HTTP requests

As we can see, clients can send requests to our backend server, and the server will respond to them. There are five commonly used methods in a REST-based architecture:

- **GET:** This is used to read resources. Generally, it should not influence the database state and, given the same input, it should return the same output (unless the database state was changed through other requests). This behavior is called **idempotence**. In response to a successful GET request, a server usually returns the resource(s) with a 200 OK status code.
- **POST:** This is used to create new resources, from the information provided in the request body. In response to a successful POST request, a server usually either returns the newly created object with a 201 Created status code or returns an empty response (with 201 Created status code) with a URL in the **Location** header that points to the newly created resource.
- **PUT:** This is used to update an existing resource with a given ID, replacing the resource completely with the new data provided in the request body. In some cases, it can also be used to create a new resource with a client-specified ID. In response to a successful PUT request, a server either returns the updated resource with a 200 OK status code, 204 No Content if it does not return the updated resource, or 201 Created if it created a new resource.
- **PATCH:** This is used to modify an existing resource with a given ID, only updating the fields specified in the request body instead of replacing the whole resource. In response to a successful PATCH request, a server either returns the updated resource with 200 OK or 204 No Content if it does not return the updated resource.
- **DELETE:** This is used to delete a resource with a given ID. In response to a successful DELETE request, a server either returns the deleted resource with 200 OK or 204 No Content if it does not return the deleted resource.

HTTP REST API routes are usually defined in a folder-like structure. It is always a good idea to prefix all routes with `/api/v1/` (`v1` being the version of the API definition, starting with `1`). If we want to change the API definition later, we can then easily run `/api/v1/` and `/api/v2/` in parallel for a while until everything is migrated.

Defining our API routes

Now that we have learned how HTTP REST APIs work, let's start by defining routes for our backend, covering all functionality we have already implemented in the service functions:

- **GET `/api/v1/posts`**: Get a list of all posts
- **GET `/api/v1/posts?sortBy=updatedAt&sortOrder=ascending`**: Get a list of all posts sorted by `updatedAt` (ascending)

NOTE

Everything after the `?` symbol is called a query string and follows the format `key1=value1&key2=value2&...`. The query string can be used to provide additional optional parameters to a route.

- **GET `/api/v1/posts?author=daniel`**: Get a list of posts by author "daniel"
- **GET `/api/v1/posts?tag=react`**: Get a list of posts with the tag `react`
- **GET `/api/v1/posts/:id`**: Get a single post by ID
- **POST `/api/v1/posts`**: Create a new post
- **PATCH `/api/v1/posts/:id`**: Update an existing post by ID
- **DELETE `/api/v1/posts/:id`**: Delete an existing post by ID

As we can see, by putting together our already developed service functions and what we have learned about REST APIs, we can easily define routes for our backend. Now that we have defined our routes, let's set up Express and our backend server to be able to expose those routes.

NOTE

This is just one example of how a REST API can be designed. It is intended as an example to get you started with full-stack development. Later, on your own time, feel free to check out other resources, such as <https://standards.rest>, to deepen your knowledge of REST API designs.

Setting up Express

Express is a web application framework for Node.js. It provides utility functions to easily define routes for REST APIs and serve HTTP servers. Express is also very extensible, and there are many plugins for it in the JavaScript ecosystem.

NOTE

While Express is the most well-known framework at the time of writing, there are also newer ones, such as Koa (<https://koajs.com>) or Fastify (<https://fastify.dev>). Koa is designed by the team behind Express but aims to be smaller, more expressive, and more robust. Fastify focuses on efficiency and low overhead. Feel free to check these out on your own time to see if they fit your requirements better.

Before we can set up the routes, let's take some time to set up our Express application and backend server by following these steps:

1. First, install the **express** dependency:

```
$ npm install express@4.18.2
```

2. Create a new **src/app.js** file. This file will contain everything needed to set up our Express app. In this file, first import **express**:

```
import express from 'express'
```

3. Then create a new Express app, as follows:

```
const app = express()
```

4. Now we can define routes on the Express app. For example, to define a GET route, we can write the following code:

```
app.get('/', (req, res) => {  
  res.send('Hello from Express!')  
})
```

5. We export the app to be able to use it in other files:

```
export { app }
```

6. Next, we need to create a server and specify a port, similar to what we did before when creating an HTTP server. To do so, we create a new **src/index.js** file. In this file, we import the Express app:

```
import { app } from './app.js'
```

7. Then, we define a port, make the Express app listen to it, and log a message telling us where the server is running:

```
const PORT = 3000
app.listen(PORT)
console.info(`express server running on http://localhost:${PORT}`)
```

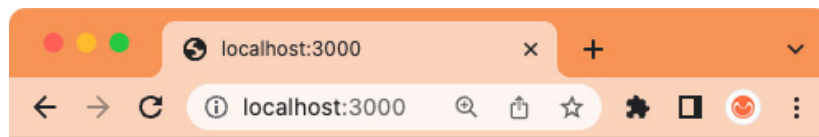
8. Edit **package.json** and add a **start** script to run our server:

```
"scripts": {
  "start": "node src/index.js",
```

9. Run the backend server by executing the following command:

```
$ npm start
```

10. Now, navigate to **http://localhost:3000/** in your browser and you will see **Hello from Express!** Being printed, just like before with the plain http server:



Hello from Express!

Figure 3.8 – Accessing our first Express app from the browser!

That's all there is to setting up a simple Express app! We can now keep defining routes by using **app.get()** for GET routes, **app.post()** for POST routes, etc. However, before we start developing our routes, let's take some time to improve our development environment. First, we should make **PORT** and **DATABASE_URL** configurable so that we can change them without having to change the code. To do so, we are going to use environment variables.

Using dotenv for setting environment variables

A good way to load environment variables is using **dotenv**, which loads environment variables from **.env** files into our **process.env**. This makes it easy to define environment variables for local development while keep-

ing it possible to set them differently in, for example, a testing environment. Follow these steps to set up **dotenv**:

1. Install the **dotenv** dependency:

```
$ npm install dotenv@16.3.1
```

2. Edit **src/index.js**, import **dotenv** there, and call **dotenv.config()** to initialize the environment variables. We should do this before we call any other code in our app:

```
import dotenv from 'dotenv'  
dotenv.config()
```

3. Now we can start replacing our static variables with environment variables. Edit **src/index.js** and replace the static port **3000** with **process.env.PORT**:

```
const PORT = process.env.PORT
```

4. We have already migrated the **initDatabase** function to use **process.env.DATABASE_URL** earlier when we set up Jest. Now, we can edit **src/index.js** and import **initDatabase** there:

```
import { initDatabase } from './db/init.js'
```

5. Adjust the existing code to first call **initDatabase**, and only when the database initialized, start the Express app. We can now also handle errors while connecting to the database by adding a try/catch block:

```
try {  
  await initDatabase()  
  const PORT = process.env.PORT  
  app.listen(PORT)  
  console.info(`express server running on http://localhost:${PORT}`)  
} catch (err) {  
  console.error('error connecting to database:', err)  
}
```

6. Finally, create a **.env** file in the root of the project and define the two environment variables there:

```
PORT=3000  
DATABASE_URL=mongodb://localhost:27017/blog
```

7. We should exclude the `.env` file from the Git repository, as it is only used for local development. Edit `.gitignore` and add `.env` to it in a new line:

```
.env
```

At the moment, we have no sensible information in our environment variables, but it is still a good practice to do this already now. Later, we may have some credentials in the environment variables, which we do not want to accidentally push to a Git repository.

8. To make it easier for someone to get started with our project, we can create a copy of our `.env` file and duplicate it to `.env.template`, making sure that it does not contain any sensitive credentials, of course! Sensitive credentials could instead be stored in, for example, a shared password manager.
9. If it is still running from before, stop the server (by pressing `Ctrl + C` in the Terminal) and start it again as follows:

```
$ npm start
```

You will get the following result:

```
○ → ~/D/F/ch3 ↵ main ± > npm start  
  
> ch3@0.0.0 start  
> node src/index.js  
  
successfully connected to database: mongodb://localhost:27017/ch3  
express server running on http://localhost:3000
```

Figure 3.9 – Initializing the database connection and the Express server with environment variables

As we can see, `dotenv` makes it easy to maintain environment variables for development while still allowing us the possibility to change them in a continuous integration, testing, or production environment.

You may have noticed that we need to manually restart the server after making some changes. This is a stark contrast to the hot reloading we got out of the box from Vite, where any changes we make are applied to the frontend in the browser instantly. Let's now spend some time to improve the development experience by making the server auto-restart on changes.

Using nodemon for easier development

To make our server auto-restart on changes, we can use the **nodemon** tool. The **nodemon** tool allows us to run our server, similarly to the node CLI command. However, it offers the possibility to auto-restart the server on changes to the source files.

1. Install the **nodemon** tool as a dev dependency:

```
$ npm install --save-dev nodemon@3.0.2
```

2. Create a new **nodemon.json** file in the root of your project and add the following contents to it:

```
{
  "watch": [".", "src", ".env", "package-lock.json"]
}
```

This makes sure that all code in the **src/** folder is watched for changes, and it will refresh if any files inside it are changed. Additionally, we specified the **.env** file in case environment variables are changed and the **package-lock.json** file in case packages are added or upgraded.

3. Now, edit **package.json** and define a new **"dev"** script that runs **nodemon**:

```
"scripts": {
  "dev": "nodemon src/index.js",
}
```

4. Stop the server (if it is currently running) and start it again by running the following command:

```
$ npm run dev
```

5. As we can see, our server is now running through **nodemon**! We can try it out by changing the port in the **.env** file:

```
PORT=3001
DATABASE_URL=mongodb://localhost:27017/blog
```

6. Edit **.env.template** as well to change the port to **3001**:

```
PORT=3001
```

7. Keep the server running.

```
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): src/**/* .env package.json
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node src/index.js`
successfully connected to database: mongodb://localhost:27017/ch3
express server running on http://localhost:3000
[nodemon] restarting due to changes...
[nodemon] starting `node src/index.js`
successfully connected to database: mongodb://localhost:27017/ch3
express server running on http://localhost:3001
```

Figure 3.10 – Nodemon automatically restarting the server after we changed the port

After making the change, **nodemon** automatically restarted the server for us with the new port. We now have something like hot reloading, but for backend development—awesome! Now that we have improved the developer experience on the backend, let's start writing our API routes with Express. Keep the server running (via **nodemon**) to see it restart and update live while coding!

Creating our API routes with Express

We can now start creating our previously defined API routes with express. We start by defining the GET routes:

1. Create a new **src/routes/posts.js** file and import the service functions there:

```
import {
  listAllPosts,
  listPostsByAuthor,
  listPostsByTag,
  getPostById,
} from '../services/posts.js'
```

2. Now create and export a new function called **postsRoutes**, which takes the Express app as an argument:

```
export function postsRoutes(app) {
```

3. In this function, define the routes. Start with the GET **/api/v1/posts** route:

```
app.get('/api/v1/posts', async (req, res) => {
```

4. In this route, we need to make use of query params (**req.query** in Express) to map them to the arguments of our functions. We want to

be able to add query params for **sortBy**, **sortOrder**, **author**, and **tag**:

```
const { sortBy, sortOrder, author, tag } = req.query
const options = { sortBy, sortOrder }
```

5. Before we call our service functions, which might throw an error if we pass invalid data to the database functions, we should add a try-catch block to handle potential errors properly:

```
try {
```

6. We now need to check if the **author** or **tag** was provided. If both were provided, we return a **400 Bad Request** status code and a JSON object with an error message by calling **res.json()**:

```
if (author && tag) {
  return res
    .status(400)
    .json({ error: 'query by either author or tag, not both' })
}
```

7. Otherwise, we call the respective service function and return a JSON response in Express by calling **res.json()**. In case an error happened, we catch it, log it, and return a 500 status code:

```
    } else if (author) {
      return res.json(await listPostsByAuthor(author, options))
    } else if (tag) {
      return res.json(await listPostsByTag(tag, options))
    } else {
      return res.json(await listAllPosts(options))
    }
  } catch (err) {
    console.error('error listing posts', err)
    return res.status(500).end()
  }
})
```

8. Next, we define an API route to get a single post. We use the **:id** param placeholder to be able to access it as a dynamic parameter in the function:

```
app.get('/api/v1/posts/:id', async (req, res) => {
```

9. Now, we can access **req.params.id** to get the **:id** part of our route and pass it to our service function:

```
const { id } = req.params
try {
  const post = await getPostById(id)
```

10. If the result of the function is **null**, we return a 404 response because the post was not found. Otherwise, we return the post as a JSON response:

```
    if (post === null) return res.status(404).end()
    return res.json(post)
  } catch (err) {
    console.error('error getting post', err)
    return res.status(500).end()
  }
})
}
```

By default, Express will return the JSON response with status 200 OK.

11. After defining our GET routes, we still need to mount them in our app. Edit **src/app.js** and import the **postsRoutes** function there:

```
import { postsRoutes } from './routes/posts.js'
```

12. Then, call the **postsRoutes(app)** function after initializing our Express app:

```
const app = express()
postsRoutes(app)
```

13. Go to <http://localhost:3001/api/v1/posts> to see the route in action!

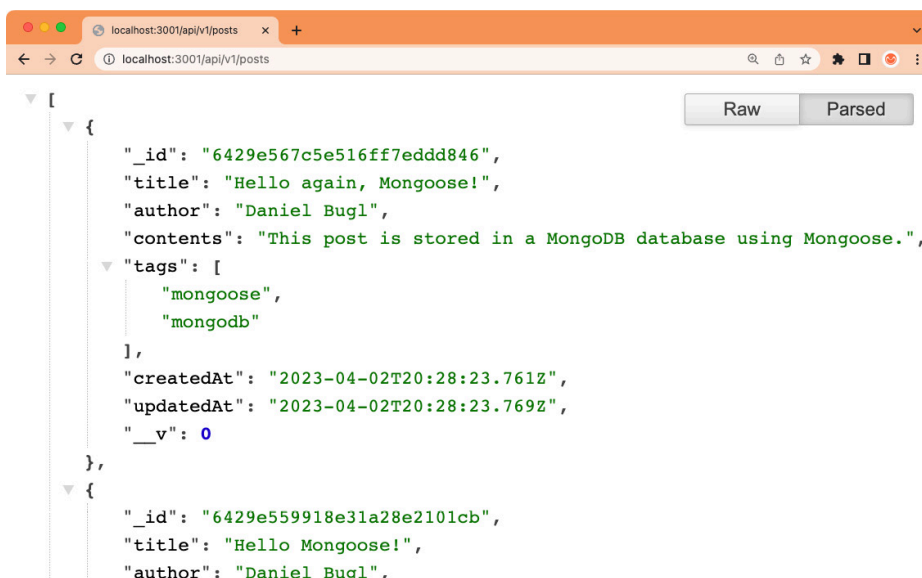


Figure 3.11 – Our first real API route in action!

TIP

*You can install a **JSON Formatter** extension in your browser to format the JSON response nicely, like in Figure 3.11.*

After defining the GET routes, we need to define the POST routes. However, these accept a body, which will be formatted as JSON objects. As such, we need a way to parse this JSON body in Express.

Defining routes with a JSON request body

To define routes with a JSON request body in Express, we need to use the **body-parser** module. This module detects if a client sent a JSON request (by looking at the **Content-Type** header) and then automatically parses it for us so that we can access the object in **req.body**.

1. Install the **body-parser** dependency:

```
$ npm install body-parser@1.20.2
```

2. Edit **src/app.js** and import the **body-parser** there:

```
import bodyParser from 'body-parser'
```

3. Now add the following code after our app is initialized to load the **body-parser** plugin as middleware into our Express app:

```
const app = express()  
app.use(bodyParser.json())
```

NOTE

*Middleware in Express allows us to do something before and after each request. In this case, **body-parser** is reading the JSON body for us, parsing it as JSON and giving us a JavaScript object that we can easily access from our route definitions. It should be noted that only routes defined after the middleware have access to it, so the order of defining middleware and routes is important!*

4. After loading the **body-parser**, we edit **src/routes/posts.js** and import the service functions needed to make the rest of our routes:

```
createPost,
```

```
    updatePost,  
    deletePost,  
  } from '../services/posts.js'
```

5. Now, we define the **POST /api/v1/posts** route by using **app.post** and **req.body**, inside of the **postsRoutes** function:

```
app.post('/api/v1/posts', async (req, res) => {  
  try {  
    const post = await createPost(req.body)  
    return res.json(post)  
  } catch (err) {  
    console.error('error creating post', err)  
    return res.status(500).end()  
  }  
})
```

6. Similarly, we can define the update route, where we need to make use of the **id** param and the request body:

```
app.patch('/api/v1/posts/:id', async (req, res) => {  
  try {  
    const post = await updatePost(req.params.id, req.body)  
    return res.json(post)  
  } catch (err) {  
    console.error('error updating post', err)  
    return res.status(500).end()  
  }  
})
```

7. Finally, we define the delete route, which does not require the **body-parser**; we just need to get the **id** param here. We return 404 if the post was not found, and 204 No Content if the post was deleted successfully:

```
app.delete('/api/v1/posts/:id', async (req, res) => {  
  try {  
    const { deletedCount } = await deletePost(req.params.id)  
    if (deletedCount === 0) return res.sendStatus(404)  
    return res.status(204).end()  
  } catch (err) {  
    console.error('error deleting post', err)  
    return res.status(500).end()  
  }  
})
```

As we can see, Express makes defining and handling routes, requests, and responses much easier. It already detects and sets headers for us, and thus it can read and send JSON responses properly. It also allows us to change the HTTP status code easily.

Now that we finished defining the routes with a JSON request body, let's allow access to our routes from other URLs using **cross-origin resource sharing (CORS)**.

Allowing access from other URLs using CORS

Browsers have a safety feature to only allow us to access APIs on the same URL as the page we are currently on. To allow access to our backend from other URLs than the backend URL itself (for example, when we run the frontend on a different port in the next chapter), we need to allow CORS requests. Let's set that up now by using the **cors** library with Express:

1. Install the **cors** dependency:

```
$ npm install cors@2.8.5
```

2. Edit **src/app.js** and import **cors** there:

```
import cors from 'cors'
```

3. Now add the following code after our app is initialized to load the **cors** plugin as middleware into our Express app:

```
const app = express()  
app.use(cors())  
app.use(bodyParser.json())
```

Now that CORS requests are allowed, we can start trying out the routes in a browser!

Trying out the routes

After defining our routes, we can try them out by using the **fetch()** function in the browser:

1. In your browser, go to **http://localhost:3001/**, open the console by right-clicking on a page and clicking **Inspect**, then go to the **Console** tab.

2. In the console, enter the following code to make a GET request to get all posts:

```
fetch('http://localhost:3001/api/v1/posts')
  .then(res => res.json())
  .then(console.log)
```

3. Now we can modify this code to make a POST request by specifying the **Content-Type** header to tell the server that we will be sending JSON and then sending a body with **JSON.stringify** (as the body has to be a string):

```
fetch('http://localhost:3001/api/v1/posts', {
  headers: { 'Content-Type': 'application/json' },
  method: 'POST',
  body: JSON.stringify({ title: 'Test Post' })
})
  .then(res => res.json())
  .then(console.log)
```

4. Similarly, we can also send a **PATCH** request, as follows:

```
fetch('http://localhost:3001/api/v1/posts/642a8b15950196ee8b3437b2', {
  headers: { 'Content-Type': 'application/json' },
  method: 'PATCH',
  body: JSON.stringify({ title: 'Test Post Changed' })
})
  .then(res => res.json())
  .then(console.log)
```

Make sure to replace the MongoDB IDs in the URL with the one returned from the **POST** request made before!

5. Finally, we can send a **DELETE** request:

```
fetch('http://localhost:3001/api/v1/posts/642a8b15950196ee8b3437b2', {
  method: 'DELETE',
})
  .then(res => res.status)
  .then(console.log)
```

6. When doing a GET request, we can see that our post has now been deleted again:

```
fetch('http://localhost:3001/api/v1/posts/642a8b15950196ee8b3437b2')
  .then(res => res.status)
  .then(console.log)
```


This request should now return a **404**.

TIP

*Instead of the browser console, you can also use command line tools such as **curl** or apps such as Postman to make the requests. Feel free to use different tools to try out the requests if you are already familiar with them.*

We have now successfully defined all routes needed to handle a simple blog post API!

Summary

The first version of our backend service is now complete, allowing us to create, read, update, and delete blog posts via a REST API (using Express), which then get stored in a MongoDB database (using Mongoose). Additionally, we have created service functions with unit tests, defined using the Jest test suite. All in all, we managed to create a solid foundation for our backend in this chapter.

In the next chapter, [**Chapter 4**](#), *Integrating a Frontend Using React and TanStack Query*, we are going to integrate our backend into a React frontend using TanStack Query, a library to handle asynchronous state and thus data fetched from our server. This means that, after the next chapter, we will have developed our first full-stack application!